LLMOps Part 1: Operating AI Systems in Production



INTERVIEW KICKSTART

Introduction – Pradeep

Senior Lead Software Engineer, AI & Data.



Full Stack Data Professional



Teaching Gen AI

Welcome to today's class, before we begin, pop into the chat:

Hannah Chen

Data Scientist



Bay Area

Grammarly, ex-Uber

{ik} INTERVIEW KICKSTART

Structure of Class

1. **Sunday Class - 4 hr (Sunday)**
2. **Assignments -** MCQ & Coding Assignment





{ik} | INTERVIEW KICKSTART

Wednesday and Thursday Sessions

Wednesday - Technical Coaching Session

One hour of topical review on the class. This is an excellent time to reinforce Sunday's content or ask advanced questions, depending on your experience.

Thursday - Assignment Review Session

Two hours to review the assignment and the MCQs. Both assignments and MCQs are designed to reinforce the learning objectives which tie directly to knowledge and skills you will need in industry and in interviewing.



Optimize Your Experience



- ✅ Interact with your instructors via Sunday live class.
- ✅ Don’t be shy to speak up and get clarifications !



- ✅ Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
- ✅ Use your resources! It’s your experience – what you put in is what you’ll get out.

{ik} INTERVIEW KICKSTART

Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- Optional **Post class videos** for additional reference
- **Technical coaching sessions** on Wednesday to clarify all your queries
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- **Support tickets** - Raise them any time you want any clarifications/ tech concerns , etc on UpLevel

Today's Agenda



1



The Ops Mindset

2



MLOps

3



LLMOps

4



Integrated Demo

{ik} INTERVIEW KICKSTART

Topic 1: Foundations

The Ops Mindset for AI Systems

What you'll learn:

- Why AI systems need a new operational model
- MLOps vs LLMOps vs AgentOps
- 7 production failure modes unique to AI
- The AI system lifecycle
- Observability beyond logs and uptime
- The 7 core AI Ops Mindset principles

{ik} INTERVIEW KICKSTART

From Software Ops to AI Ops

"Traditional software follows deterministic logic. AI systems behave probabilistically — this shift changes everything about how we operate them."

What Changes in AI Ops?

- Testing — no more exact-match assertions
- Monitoring — statistical signals, not binary
- Failure detection — gradual, not immediate
- Observability — trace inputs, prompts & outputs

The New Stack

- **MLOps** → ML model lifecycle
- **LLMOps** → Prompts · Tokens · Guardrails
- **AgentOps** → Tools · Memory · Orchestration

Dimension	Traditional Software	AI Systems
Behavior	Deterministic	Probabilistic
Output	Exact, repeatable	Variable, contextual
Failures	Clear, binary errors	Silent, gradual degradations
Testing	Pass/fail assertions	Distribution-based scoring

{ik} INTERVIEW KICKSTART

Deterministic Software vs Probabilistic AI

"In deterministic systems, bugs are binary. In AI systems, failures are gradual, contextual, and data-dependent."

Key Concepts to Internalize

- **Variance** — the same input produces slightly different outputs on each call
- **Non-repeatability** — you cannot replay a request and get the exact same result
- **Drift** — model behavior degrades over time without any code change
- **Emergent behavior** — unexpected outputs from context, data, or prompt interactions

Deterministic Software	AI Systems
Rule-based logic	Statistical inference
Exact, repeatable output	Variable outputs
Unit testable	Behavior-based testing
Clear, binary failures	Silent, gradual degradations
Stable over time	Drifts over time

A model can be 100% available and 40% useful. Uptime ≠ correctness. Monitoring must be statistical, not binary.

{ik} INTERVIEW KICKSTART

Why MLOps ≠ LLMOps ≠ AgentOps

"Each layer adds more unpredictability, more statefulness, and more failure points."

MLOps

ML Model Lifecycle

- Data pipelines & feature stores
- Model training & hyperparameters
- Model registry & versioning
- Deployment & rollback automation

LLMOps

Generative Model Operations

- Prompt management & versioning
- Context window optimization
- Token cost monitoring & budgets
- Guardrails & safety layers
- Evaluation pipelines

AgentOps

Autonomous Tool-Using Systems

- Multi-step workflow orchestration
- Tool calling reliability & retries
- Memory & state management
- Multi-agent coordination



{ik} INTERVIEW KICKSTART

New Production Failure Modes in AI Systems

"Unlike traditional 500 errors, AI failures often look plausible, pass superficial checks, and only fail under specific user patterns."

The 7 Failure Modes

- **Data Drift:** Input distribution shifts; model silently degrades
- **Concept Drift:** Target variable changes meaning over time
- **Prompt Drift:** Small prompt edits cause quality regressions
- **Tool Failure:** Agent tool calls timeout or return bad data
- **Hallucination:** Confident but factually incorrect output
- **Cost Explosion:** Token runaway loops in agent chains
- **Latency Amplification:** Chained LLM calls multiply end-to-end latency

Why These Are Dangerous

- Look correct and plausible to the user
- Pass shallow automated checks
- Only manifest under specific conditions
- Accumulate silently before becoming visible

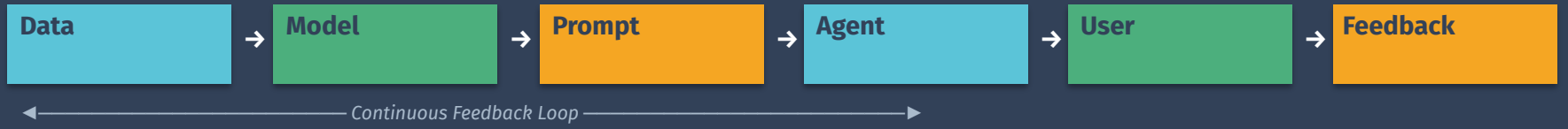
Observability Must Include

- Input logging — what came in
- Prompt logging — what was sent to the model
- Response logging — what came back
- Tool trace logging — external calls & results

{ik} INTERVIEW KICKSTART

The AI System Lifecycle

"AI systems are cyclical. Unlike static apps, they continuously evolve — operational maturity means controlling that loop."



Expanded Phase View

Phase	What Happens
Data Collection	Gather from user interactions, documents, APIs
Data Processing	Clean, label, chunk, and embed
Model Training	Fine-tune or pre-train where applicable
Model Deployment	Serve with versioning, canary, and rollback
Prompt Engineering	Author, test, version, and deploy prompts
Agent Orchestration	Assemble tools, memory, and routing logic
User Interaction	Serve real traffic with guardrails active
Feedback Loop	Capture ratings, corrections, behavioral signals

Observability in AI Systems

"Uptime ≠ correctness. You must monitor behavior, not just availability."

Traditional vs AI Observability

Traditional	AI Observability
Logs	Prompt version tracking
Metrics (latency, errors)	Model response scoring
Distributed traces	Human feedback capture
—	Cost tracking per request
—	Latency per tool call
—	Drift monitoring over time

5 Dimensions You Must Monitor

- **Quality:** Is the output correct, relevant, on-topic?
- **Consistency:** Does the same input class produce similar quality?
- **Cost:** Token spend per request, session, and day?
- **Safety:** Are guardrails catching what they should?
- **Tool Reliability:** Are external tool calls within latency SLAs?

{ik} INTERVIEW KICKSTART

The AI Ops Mindset

"AI systems are living systems. Operate them like adaptive organisms, not static software."

7 Core Principles

- **Expect Uncertainty:** Design for variance from day one — not as an afterthought
- **Instrument Everything:** Logs, traces, and evals before you go to production
- **Version Prompts Like Code:** No prompt change without a version, test, and eval gate
- **Treat Agents as Distributed:** Apply retries, timeouts, and circuit breakers
- **Build Eval Pipelines Early:** Evals are your test suite — build them before you need them
- **Assume Drift Will Happen:** Schedule retraining reviews; monitor for input & concept drift
- **Observability Before Scale:** Far cheaper to instrument early than to retrofit later

You are not shipping software. You are shipping a probabilistic system that must be governed, monitored, and continuously improved.

{ik} INTERVIEW KICKSTART

Let’s see why LLMOps is needed

Demo Time!

{ik} INTERVIEW KICKSTART

Today's Agenda



1



The Ops Mindset

2



MLOps

3



LLMOps

4



Integrated Demo

{ik} INTERVIEW KICKSTART

Topic 2: MLOps Essentials

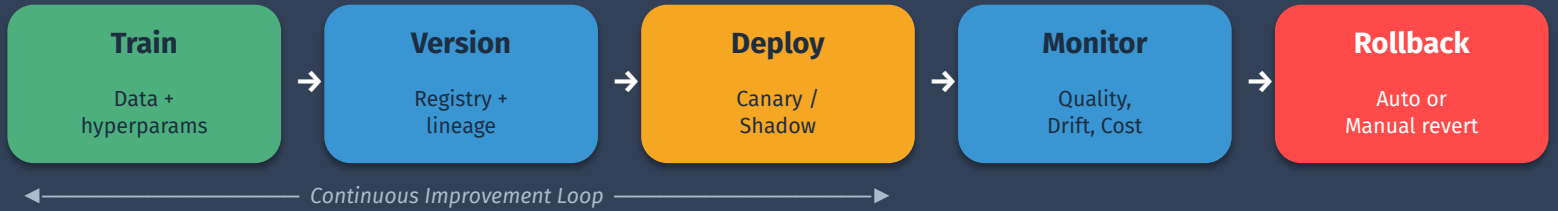
What you'll learn:

- The ML model lifecycle end-to-end
- Offline vs Online evaluation — when to use each
- Model metrics vs business metrics — the PM role
- Data drift and concept drift — silent failure modes
- Model monitoring and alert tier design
- Shadow, canary, and rollback release strategies

{ik} | INTERVIEW KICKSTART

Model Lifecycle Management

"Models are not one-time artifacts — they have a lifecycle. Every model in production must have a clear lineage."



What Each Phase Means

- **Train** — Produce a model artifact from data and hyperparameters
- **Version** — Register with full lineage: data snapshot, config, eval results
- **Deploy** — Serve with canary, shadow, or blue/green strategy
- **Monitor** — Track quality, drift, latency, and cost in production
- **Rollback** — Revert to previous version when metrics degrade

The Model Registry

e.g., MLflow, Weights & Biases — single source of truth for all artifacts.

- Every trained version + training data snapshot
- Hyperparameters and evaluation results
- Deployment history and current production status

Rollback should be as easy as deploying. If a new model degrades, you must revert within minutes — not days.

{ik} INTERVIEW KICKSTART

Offline vs Online Evaluation

"Offline eval is your safety net. Online eval is your reality check. A mature ML team runs both."

OFFLINE (Lab / CI-CD)

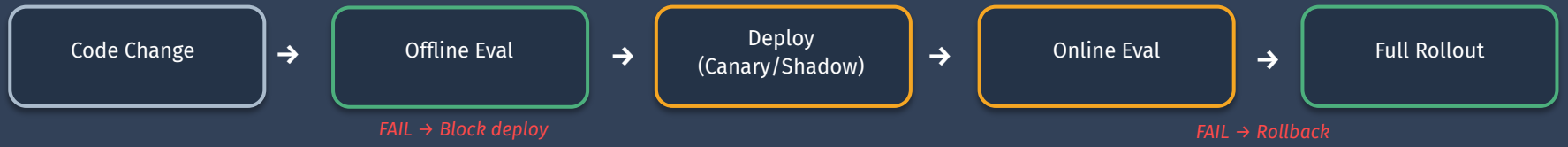
- Held-out test sets — before deployment
- Precision, Recall, F1, BLEU, ROUGE
- Safe, repeatable, cheap · Regression gate

VS

ONLINE (Production Traffic)

- A/B tests · Shadow scoring · Interleaving
- Business KPIs, engagement, revenue
- Real user behaviour · Validated in the wild

How They Work Together in a Pipeline



Dimension	Offline Evaluation	Online Evaluation
When	Before deployment	In production with real traffic
Methods	Held-out test sets	A/B tests, shadow scoring, interleaving
Pros	Safe, repeatable, cheap	Captures real user behaviour
Cons	May not reflect real-world distribution	Risk of degraded user experience

{ik} INTERVIEW KICKSTART

Model Metrics vs Business Metrics

"Technical teams optimize model scores. Product teams care about business outcomes. PMs and TPMs must own the bridge between them."

Model Performance Metrics		Business Metrics
Accuracy, F1, AUC-ROC		Revenue impact, conversion rate
Latency — p50, p95, p99	↔	User satisfaction — NPS, CSAT
Throughput (requests / sec)		Retention and engagement
Precision / Recall		Task completion rate

The Gap — Why This Matters

- A model can improve on F1 and simultaneously hurt business KPIs
- Test set distribution ≠ real user distribution
- Optimizing precision can degrade recall for edge cases
- Model correctness ≠ user-perceived quality

The PM / TPM / EM Role

- Define upfront: which business metric does this model serve?
- Establish the mapping: model metric X → business outcome Y
- Tie evaluation pipelines to business metrics, not just model scores
- **Always ask:** does a better model score mean a better user experience?

KEY INSIGHT: A 2% improvement in F1 means nothing if it does not move the needle on conversion or retention. Always tie model evaluation to a business outcome.

{ik} INTERVIEW KICKSTART

Data Drift vs Concept Drift

"Neither data drift nor concept drift produces an error. The model keeps serving — confidently wrong."

DATA DRIFT

Input feature distribution changes over time



CONCEPT DRIFT

Relationship between input and target changes

BEFORE: "relevant" = keyword match
AFTER: "relevant" = semantic intent
Same input → different correct answer

	Data Drift	Concept Drift
What changes	Input feature distribution	Input-to-target relationship
Detection	KS test, PSI on input features	Monitor model performance over time
Example	Seasonal patterns, new demographics	What 'relevant' means evolves
Question	Is the data changing?	Is the world changing?

{ik} INTERVIEW KICKSTART

Data Drift vs Concept Drift

"Neither data drift nor concept drift produces an error. The model keeps serving — confidently wrong."

Mitigation Strategies

- **Scheduled retraining** — Periodically retrain models using recent data to reflect changing user behavior and real-world patterns. Helps maintain accuracy and prevents performance degradation caused by outdated training distributions.
- **Drift alerts** — Continuously monitor feature distributions and prediction outputs using statistical tests. Trigger alerts when deviations exceed thresholds, enabling early investigation before business impact occurs.
- **Human-in-the-loop review** — Route low-confidence or high-risk predictions to human experts for validation. Improvs reliability, captures edge cases, and generates high-quality feedback data for future model improvement.
- **Canary evaluation** — Deploy a challenger model alongside production on limited traffic. Compare performance metrics safely before full rollout, reducing risk while validating improvements under real usage conditions.

{ik} INTERVIEW KICKSTART

Model Monitoring and Alerts

"Monitoring an ML model is not the same as monitoring an API. You need statistical awareness, not just binary health checks."

What to Monitor

Signal	What It Tells You
Prediction distribution	Are output scores shifting from baseline?
Latency — p50, p95, p99	Are SLAs being met? Tail latency spikes?
Throughput	Is the model handling expected volume?
Error rates & fallbacks	How often is the model failing or bypassed?
Input feature distributions	Are inputs drifting from training data?
Confidence scores	Are predictions becoming less certain?

PM / EM Dashboard — 3 Key Panels

Model Health Score

Single composite signal — Green / Amber / Red

SLA Compliance

Latency & uptime vs agreed targets

Drift Indicators

Input & prediction distribution trends over time

Alert Severity Tiers — Avoiding Alert Fatigue

P0 — **Model down / accuracy freefall**

Immediate rollback — page on-call now

P1 — **Drift detected / SLA breach**

Investigate within 1 hour

P2 — **Gradual degradation trend**

Schedule review within 24 hours

KEY INSIGHT: Design alerts that are actionable. Every alert needs a clear owner and a playbook. An alert nobody acts on is noise.

{ik} INTERVIEW KICKSTART

Release Strategies: Shadow / Canary / Rollback

"Release strategy is how you manage risk when shipping a new model. Always have rollback ready."

Shadow
Safest

100% → Current Model (served)

100% → New Model (logged only)

- Zero user impact
- Full production data for comparison
- Use for: high-risk changes, new architectures

Canary
Controlled Exposure

95% → Current Model

5% → New Model (ramp up gradually)

- Small real exposure — limited blast radius
- Ramp: 5% → 25% → 50% → 100%
- Use for: iterative quality improvements

Blue / Green
Full Swap

Blue (current) → standby

Green (new) → 100% traffic

- Instant traffic switch + instant rollback
- Two full environments running in parallel
- Use for: major version upgrades

Rollback — Always Enabled

Automated Metric threshold breach → auto-revert to previous registry version

Manual On-call team reverts via model registry in < 5 minutes

Decision Guide

High-risk change? → **Shadow first**

Iterative improvement? → **Canary rollout**

Major version swap? → **Blue / Green**

Any deployment? → **Rollback always enabled**

{ik} INTERVIEW KICKSTART

Today's Agenda

1



The Ops Mindset

2



MLOps



3



LLMOps

4



Integrated Demo

{ik} INTERVIEW KICKSTART

Topic 3: LLMOps Core Topics

What you'll learn:

- Prompt lifecycle and versioning strategies
- Model selection and intelligent routing
- Token economics and cost/latency/quality trade-offs
- Exact and semantic caching for LLM calls
- Guardrails, moderation, and policy enforcement
- Human-in-the-loop workflow patterns
- LLM observability: logs, traces, and evals

{ik} INTERVIEW KICKSTART

Prompt Lifecycle and Prompt Versioning

"Prompts are the new code in LLM systems. A prompt change can silently degrade quality across thousands of requests."



Version Control Strategies

Approach	How It Works	Best For
Git-based	Prompts in repo alongside code	Small teams
Prompt registry	Langfuse, PromptLayer track	Production, multi-team
Custom solution	Internal DB with metadata	Regulated / proprietary

The Prompt-Eval Pairing Rule

- Every prompt version tied to an evaluation run
- No prompt goes to production without a passing eval gate
- Creates an auditable quality record at every version

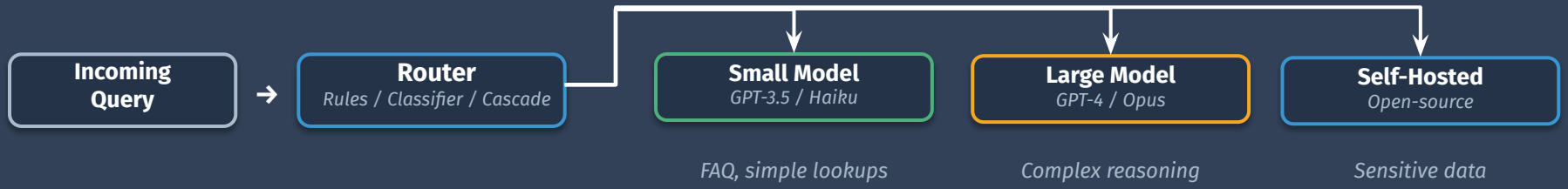
Rollback

- Revert to last known-good prompt on quality degradation
- One-click rollback — same as rolling back a code deploy
- Registry stores prompt artifacts alongside model artifacts

{ik} INTERVIEW KICKSTART

Model Selection and Routing Strategies

"Using GPT-4 for every request is like shipping everything overnight — expensive and unnecessary. Route intelligently."



Routing Criteria

- **Complexity** — how sophisticated is the reasoning required?
- **Cost budget** — per-request or per-session spend limit
- **Latency SLA** — does the use case require sub-second response?
- **Data sensitivity** — can the query be sent to an external API?

Routing Strategies

- **Static rules** — if category == X, use model Y
- **Classifier-based** — lightweight model classifies, then routes
- **Cascading** — try cheap first, escalate if confidence is low

Fallback Chains

- Primary fails → Fallback model → Graceful degradation
- Never a hard failure — always serve something useful

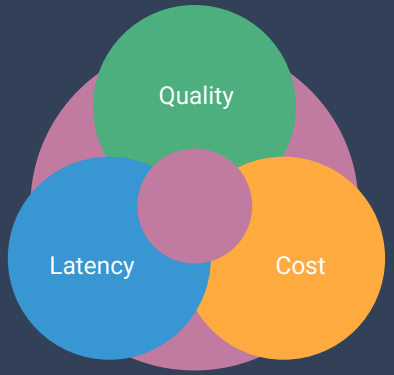
Key Insights

Cascading is the most cost-effective pattern — start cheap, escalate only when needed. Always have a fallback: the system should degrade gracefully, not fail completely.

{ik} INTERVIEW KICKSTART

Latency / Cost / Quality Trade-offs

"Every LLM call has a cost. At scale, small inefficiencies compound fast. Instrument cost per request from day one."



Optimize for two — rarely all three.

Token Economics

Concept	What It Means
Input tokens	Tokens in the prompt; billed per token
Output tokens	Model's response; typically priced higher
Context window	Max token limit — longer = more cost + latency
System prompt length	Verbose prompts at 1M req/day = thousands in extra tokens

Cost Levers — What You Can Control

- **Model tier selection** — 10–100x cost difference
- **Prompt length optimization** — concise system prompts
- **Max output token limits** — prevent runaway generation
- **Caching + Batching** — repeated queries served from cache

Budget Controls

- Per-request cost caps — fail fast if threshold exceeded
- Daily / monthly spend limits — hard billing stops
- Cost spike alerting — loops, retries, traffic surges

{ik} INTERVIEW KICKSTART

Caching Strategies: Exact and Semantic

"Caching is the single biggest lever for reducing LLM costs. Exact caching is easy and risk-free — start there."

EXACT CACHING

- Identical input → return cached output
- Near-zero latency on hit · Very low risk
- Lower hit rate — inputs must match exactly
- Start here — easy win, no downsides

SEMANTIC CACHING

- Similar-meaning input → cached output via embedding similarity
- Higher hit rate — catches paraphrased queries
- Needs careful threshold tuning
- Add after exact caching is in place

Cache Hit / Miss Flow



Where to Cache

- **Prompt-level** — full prompt string match
- **Completion-level** — repeated query patterns
- **Tool-result-level** — external API / tool calls

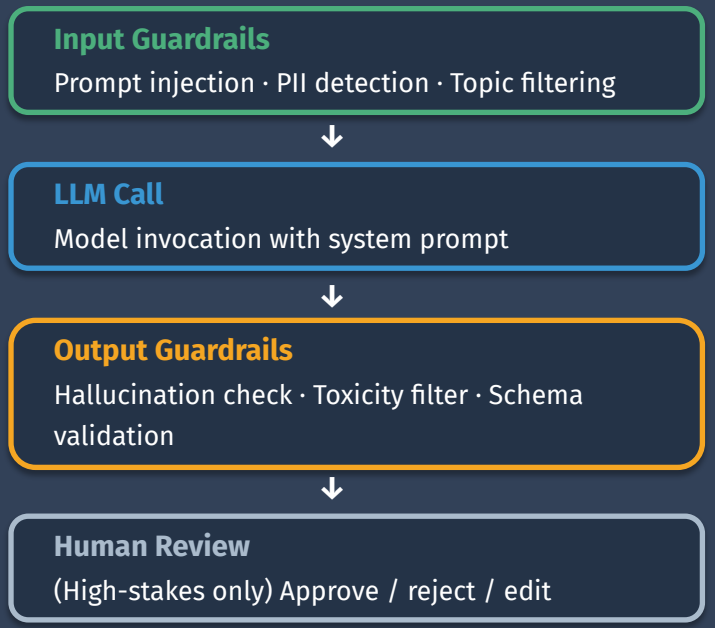
Cache Invalidation

- TTL-based expiry
- On prompt version change
- On model update

{ik} INTERVIEW KICKSTART

Guardrails / Moderation / Policy Enforcement

"Guardrails are not optional in production LLM systems. Without them, you are one user query away from a headline."



Input Guardrails

- **Prompt injection detection** — block override attempts
- **PII detection & redaction** — strip names, emails, accounts
- **Topic & intent filtering** — reject off-topic or harmful queries

Output Guardrails

- **Hallucination detection** — fact-check against source docs
- **Toxicity & bias filters** — screen harmful content
- **Schema validation** — verify JSON structure, required fields

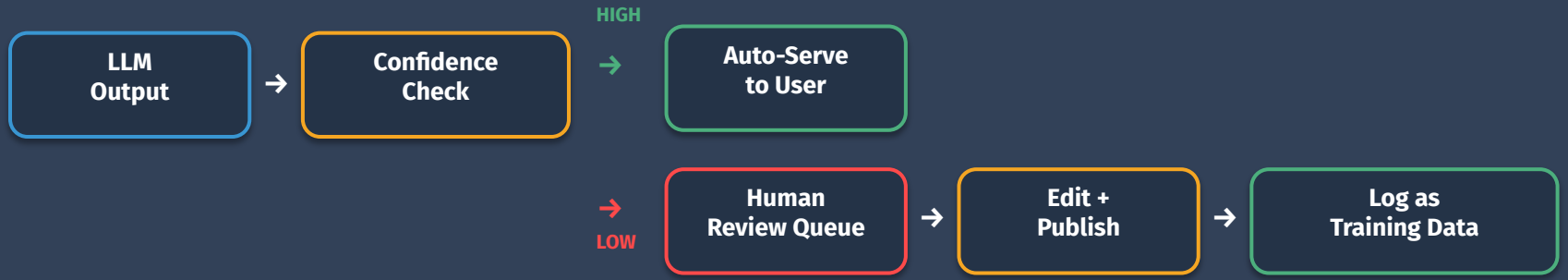
Policy Enforcement

- Industry compliance — HIPAA, SOX, GDPR
- Organizational content policies — brand voice, restricted topics
- Data residency — queries never leave specific regions

{ik} INTERVIEW KICKSTART

Human-in-the-Loop Workflows

"Not everything should be fully automated from day one. HITL is how you build trust in an LLM system incrementally."



Three HITL Workflow Patterns

Pattern	How It Works	Best For
Approve / reject	Human reviews before output is served	High-stakes, zero tolerance
Edit and confirm	Human corrects model output, then publishes	Content gen, customer-facing copy
Escalation queue	Auto-serve high-conf; queue low-conf for review	Scale efficiency + quality backstop

{ik} INTERVIEW KICKSTART

Human-in-the-Loop Workflows

"Not everything should be fully automated from day one. HITL is how you build trust in an LLM system incrementally."

When HITL Is Required

- High-stakes decisions (medical, legal, financial)
- Low-confidence model outputs below threshold
- Edge cases outside training distribution
- New domain launches with limited eval data

Feedback Loop + Maturity

- Every human correction feeds back into eval datasets
- Corrections improve prompt tuning and scoring
- Start with more oversight, automate as trust grows
- Goal: direct human effort to where it adds most value

KEY INSIGHT: Design workflows so humans review the right things — not everything. Every correction that feeds back into your eval set makes the model better over time. HITL is an investment in quality.

{ik} INTERVIEW KICKSTART

LLM Observability: Logs / Traces / Evals

"You cannot improve what you cannot see. Logging gives you the raw data. Tracing shows you the flow. Evals tell you if the output is good."

Evals
Quality scores · Regression detection
Human ratings · Automated scoring

Traces
End-to-end request flow across agents
Spans: LLM → Retrieval → Tool → Post-process

Logs
Prompt + completion pairs
Model, tokens, cost, latency, session IDs

▲ *foundation → flow → quality* ▲

Evaluation — Automated and Human

Eval Type	Methods	Signals
Automated	Relevance, faithfulness, toxicity	Continuous, no human cost
Human	Thumbs up/down, Likert, free-text	Ground truth for quality
Regression	Compare across prompt / model versions	Catch drops before users

Quality Trend
Eval scores over time

Cost / Request
Rolling avg + anomaly

Latency %iles
p50, p95, p99

Failure Rates
Guardrail + tool failures

{ik} INTERVIEW KICKSTART

LLM Observability: Logs / Traces / Evals

"You cannot improve what you cannot see. Logging gives you the raw data. Tracing shows you the flow. Evals tell you if the output is good."

Logging — What to Capture Per Request

Field	Why It Matters
Prompt text + version	Regression detection on prompt changes
Completion text	Raw material for eval scoring
Model name / version	Attribute quality to model versions
Token counts (in/out)	Cost accounting per request
Latency (total + per step)	SLA compliance & bottleneck ID
Session / user ID	Behavioural analysis across sessions

KEY INSIGHT: Invest in observability infrastructure early — retrofitting later is painful and expensive. Any degradation in quality, cost, or latency should be detectable within minutes, not days.

{ik} INTERVIEW KICKSTART

Today's Agenda

1



The Ops Mindset

2



MLOps

3



LLMOps

4



Integrated Demo

{ik} INTERVIEW KICKSTART

Let’s look at an Integrated Demo

Demo Time!

{ik} INTERVIEW KICKSTART



{ik} | INTERVIEW KICKSTART